Listing 7.6 Using default values for parameters in endpoint handlers

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

app.MapGet("/stock", StockWithDefaultValue);                    ❶

app.Run();

string StockWithDefaultValue(int id = 0) => $"Received {id}";   ❷
```

❶ The local function StockWithDefaultValue is the endpoint handler.

❷ The id parameter binds to the query string value if it's available; otherwise, it has the value 0.

We've thoroughly covered the differences between simple types and complex types and how they bind. In section 7.6 we look at some special types that don't follow these rules.

7.6 Binding services and special types

In this section you'll learn how to use some of the special types that you can bind to in your endpoint handlers. By *special*, I mean types that ASP.NET Core is hardcoded to understand or that aren't created from the details of the request, by contrast with the binding you've seen so far. The section looks at three types of parameters:

- Well-known types—that is, hard-coded types that ASP.NET Core knows about, such as `HttpContext` and `HttpRequest`
- `IFormFileCollection` and `IFormFile` for working with file uploads
- Application services registered in `WebApplicationBuilder.Services`